



TRABAJO FINAL

TALLER DE CONSTRUCCION DE SOFTWARE INTEGRANTES:

Cuberli Roman 45352533

Rinaldi Nicolas 45403417

Aguilar Esteban 46307516

Bruno Enzo Eduardo 38410678

1. Introducción

EcoMarket es una aplicación web desarrollada con el objetivo de simular el funcionamiento básico de una tienda online. El sistema permite consultar un catálogo de productos, administrar un carrito de compras y registrar órdenes una vez que el usuario confirma la compra. El proyecto fue realizado utilizando una arquitectura cliente-servidor, separando claramente el frontend del backend para facilitar el desarrollo y el mantenimiento de la aplicación.

Para la implementación del backend se utilizó Spring Boot, aprovechando su organización por capas y las herramientas que ofrece para el desarrollo de APIs REST. El frontend fue desarrollado con React, permitiendo construir una interfaz dinámica desde la cual el usuario interactúa con el sistema mediante solicitudes HTTP. La persistencia de los datos se realizó utilizando Spring Data JPA, lo que permitió trabajar con las entidades del proyecto sin necesidad de escribir consultas SQL para las operaciones más comunes.

Durante el desarrollo se buscó aplicar los conceptos trabajados a lo largo de la materia, principalmente la separación de responsabilidades, la reutilización de código y la organización del proyecto mediante capas independientes. De esta manera, cada componente cumple una función específica dentro del sistema, simplificando futuras modificaciones y favoreciendo la lectura del código.

A diferencia de una aplicación de comercio electrónico completa, EcoMarket se concentra en implementar el flujo principal de una compra. El usuario puede consultar los productos disponibles, agregarlos a un carrito, modificar las cantidades seleccionadas o eliminar artículos antes de confirmar la operación. Una vez finalizado este proceso, el sistema registra una orden utilizando la información almacenada en el carrito.

El presente informe describe la estructura general del proyecto, las tecnologías utilizadas y el funcionamiento de los principales módulos implementados durante el desarrollo. El objetivo es documentar las decisiones adoptadas y explicar de qué manera interactúan los distintos componentes que conforman la aplicación.

2. Objetivos

2.1 Objetivo general

El objetivo principal del proyecto fue desarrollar una aplicación web que simulara el funcionamiento básico de una tienda online, permitiendo al usuario consultar un catálogo de productos, administrar un carrito de compras y confirmar una orden mediante una interfaz intuitiva. Además de implementar las funcionalidades solicitadas, se buscó aplicar los conceptos vistos durante la cursada, especialmente aquellos relacionados con el desarrollo de aplicaciones web utilizando una arquitectura cliente-servidor.

2.2 Objetivos específicos

Para alcanzar el objetivo general fue necesario cumplir una serie de objetivos específicos durante el desarrollo del proyecto.

En primer lugar, se implementó un backend utilizando Spring Boot encargado de exponer una API REST con los distintos servicios utilizados por la aplicación. Esta API permite gestionar los productos disponibles, administrar el carrito de compras y registrar las órdenes generadas por los usuarios.

Por otra parte, se desarrolló una interfaz web utilizando React, desde la cual el usuario puede interactuar con el sistema de forma sencilla. La comunicación entre ambas partes de la aplicación se realiza mediante solicitudes HTTP, intercambiando información en formato JSON.

Otro de los objetivos fue aplicar una arquitectura organizada por capas, separando la lógica de presentación, la lógica de negocio y el acceso a los datos. Esta decisión permitió obtener un proyecto más ordenado, facilitando tanto el mantenimiento del código como la incorporación de nuevas funcionalidades.

Finalmente, se buscó utilizar herramientas habituales dentro del desarrollo de software, como Git para el control de versiones, GitHub para el trabajo colaborativo y Postman para realizar las pruebas de los distintos endpoints implementados durante el desarrollo.

3. Descripción general del sistema

EcoMarket fue desarrollado como una aplicación web de arquitectura cliente-servidor cuyo objetivo es simular el proceso de compra de una tienda online. El sistema está compuesto por dos partes principales: un backend desarrollado con Spring Boot, encargado de la lógica de negocio y la persistencia de los datos, y un frontend desarrollado con React, responsable de la interacción con el usuario.

El funcionamiento de la aplicación comienza cuando el usuario accede al catálogo de productos disponible desde la interfaz principal. A partir de esa información puede seleccionar los artículos que desea comprar y agregarlos a un carrito. Durante este proceso es posible modificar la cantidad de cada producto o eliminar aquellos que ya no se desean adquirir, manteniendo siempre actualizado el contenido del carrito.

Una vez finalizada la selección de productos, el usuario puede confirmar la compra. En ese momento el backend procesa la información recibida, valida el contenido del carrito y genera una orden que representa la compra realizada. De esta manera, el sistema mantiene una separación entre el carrito, utilizado de forma temporal mientras el usuario prepara la compra, y las órdenes, que representan las operaciones ya confirmadas.

Desde el punto de vista del desarrollo, se optó por utilizar una arquitectura organizada por capas. Los controladores reciben las solicitudes provenientes del frontend, los servicios implementan la lógica de negocio y los repositorios administran el acceso a la base de datos mediante Spring Data JPA. Esta organización permitió mantener una estructura clara y facilitar tanto el mantenimiento del código como la incorporación de nuevas funcionalidades.

Durante el desarrollo también se utilizaron DTO para intercambiar información entre el cliente y el servidor, evitando exponer directamente las entidades de la base de datos. Asimismo, se implementó un manejador global de excepciones para centralizar el tratamiento de errores y mantener respuestas consistentes frente a situaciones inesperadas.

En conjunto, estas decisiones permitieron obtener una aplicación organizada, donde cada componente cumple una responsabilidad específica y la comunicación entre el frontend y el backend se realiza de manera ordenada mediante una API REST.

4. Tecnologías utilizadas y arquitectura del proyecto

Para el desarrollo de EcoMarket se utilizaron distintas tecnologías que permitieron implementar una aplicación web organizada y fácil de mantener. La elección de cada una respondió a las necesidades del proyecto y a los contenidos trabajados durante la cursada.

El backend fue desarrollado utilizando **Java** junto con **Spring Boot**, framework que permitió implementar la API REST de forma organizada mediante una arquitectura por capas. Gracias a esta estructura fue posible separar claramente las responsabilidades del sistema, dejando la comunicación con el cliente a cargo de los controladores, la lógica de negocio en los servicios y el acceso a la base de datos en los repositorios.

Para la persistencia de la información se utilizó **Spring Data JPA**, apoyándose en Hibernate como herramienta de mapeo objeto-relacional. Esto permitió trabajar directamente con entidades Java y reducir considerablemente la cantidad de código necesario para realizar operaciones sobre la base de datos.

La interfaz gráfica fue desarrollada utilizando **React**, acompañado por **Vite** como herramienta de desarrollo. Esta combinación permitió construir una aplicación dinámica capaz de consumir los servicios expuestos por el backend mediante solicitudes HTTP y actualizar la información mostrada al usuario sin necesidad de recargar la página.

Durante el desarrollo también se utilizaron **Git** y **GitHub** para el control de versiones y el trabajo colaborativo entre los integrantes del grupo. De esta manera fue posible distribuir las tareas, mantener un historial de cambios y unificar el desarrollo mediante un repositorio compartido.

Para verificar el correcto funcionamiento de los distintos endpoints implementados en la API se utilizó **Postman**, herramienta que permitió probar cada servicio antes de integrarlo con el frontend.

Desde el punto de vista de la arquitectura, el proyecto sigue un modelo por capas. El frontend desarrollado en React envía solicitudes HTTP hacia el backend, donde los controladores reciben las peticiones y las derivan a los servicios correspondientes. Estos últimos implementan la lógica de negocio y, cuando es necesario acceder a la información almacenada, utilizan los repositorios proporcionados por Spring Data JPA.

Finalmente, los datos recuperados son enviados nuevamente al frontend en formato JSON para ser representados en la interfaz.

La siguiente figura resume el funcionamiento general de la aplicación:

Frontend (React)



Controllers



Services



Repositories



Base de datos

Esta organización permitió mantener una estructura clara durante todo el desarrollo del proyecto y facilitó tanto la reutilización del código como el mantenimiento de las distintas funcionalidades implementadas.

5. Desarrollo del Backend

El backend de EcoMarket fue desarrollado utilizando Spring Boot y organizado mediante una arquitectura por capas. Esta estructura permitió dividir el proyecto en distintos paquetes,

donde cada uno cumple una función específica dentro de la aplicación. De esta manera se logró una mejor organización del código, facilitando tanto su mantenimiento como la incorporación de nuevas funcionalidades.

La capa **Controller** representa el punto de entrada de todas las solicitudes realizadas desde el frontend. Cada controlador recibe las peticiones HTTP, obtiene la información enviada por el cliente y la deriva al servicio correspondiente. Durante el desarrollo se evitó incorporar lógica de negocio dentro de estas clases, manteniendo únicamente las tareas relacionadas con la comunicación entre la aplicación y el usuario.

La capa **Service** concentra las reglas de negocio del sistema. Es aquí donde se implementan las operaciones necesarias para administrar los productos, gestionar el carrito de compras y registrar las órdenes. Centralizar esta lógica permitió reutilizar código y mantener una separación clara entre el procesamiento de la información y la comunicación con el cliente.

El acceso a la base de datos se realiza mediante la capa **Repository**, implementada utilizando Spring Data JPA. Gracias a esta herramienta fue posible realizar las operaciones de consulta, creación, actualización y eliminación de registros sin necesidad de escribir manualmente consultas SQL para las operaciones más habituales.

Dentro del paquete **model** se encuentran las entidades principales del proyecto. Estas clases representan la información persistida en la base de datos y establecen las relaciones necesarias entre los distintos elementos del sistema. Las entidades más importantes son **Producto**, **Carrito**, **ItemCarrito** y **Orden**, las cuales intervienen directamente en el flujo de compra implementado por la aplicación.

Para la comunicación entre el frontend y el backend también se implementaron distintos **DTO (Data Transfer Object)**, como **AgregarItemRequest**, **ModificarItemRequest** y **ConfirmarOrdenRequest**. Estos objetos permiten transportar únicamente la información necesaria para cada operación, evitando exponer directamente las entidades de la base de datos y manteniendo desacopladas ambas capas de la aplicación.

Por último, el proyecto incorpora un **GlobalExceptionHandler**, encargado de centralizar el manejo de excepciones producidas durante la ejecución de la API.

Gracias a este componente fue posible mantener respuestas consistentes frente a distintos tipos de errores y evitar que cada controlador tuviera que implementar su propio tratamiento de excepciones.

6. Funcionamiento del sistema

El funcionamiento de EcoMarket se basa en la interacción entre el frontend desarrollado en React y la API REST implementada con Spring Boot. Todas las acciones realizadas por el usuario desde la interfaz generan solicitudes HTTP que son procesadas por el backend, donde se ejecuta la lógica correspondiente y se devuelve una respuesta en formato JSON.

El flujo de uso comienza cuando el usuario accede a la aplicación y consulta el catálogo de productos disponible. Para ello, el frontend realiza una petición al módulo de productos, que recupera la información almacenada en la base de datos y la devuelve para ser representada en pantalla. De esta manera, el usuario puede visualizar los artículos disponibles antes de iniciar una compra.

Una vez seleccionado un producto, la aplicación permite incorporarlo al carrito de compras. Esta operación es procesada por el módulo del carrito, que recibe la información enviada por el frontend mediante un objeto DTO y delega el procesamiento a la capa de servicios. Allí se verifica la existencia del producto, se actualiza el contenido del carrito y se recalcula automáticamente el importe correspondiente a la compra.

Mientras el carrito permanece activo, el usuario puede modificar la cantidad de cualquiera de los productos agregados o eliminar aquellos que ya no desea adquirir. Cada una de estas operaciones sigue el mismo recorrido dentro de la aplicación: el controlador recibe la solicitud, el servicio aplica las reglas de negocio y el repositorio actualiza la información almacenada en la base de datos.

Cuando el usuario decide finalizar la compra, el frontend envía una nueva solicitud al módulo de órdenes. A partir de la información contenida en el carrito, el sistema genera un orden que representa la compra realizada. Esta separación entre carrito y orden permite diferenciar claramente el proceso de selección de productos de la operación final ya confirmada.

Durante todo este recorrido se mantiene una distribución clara de responsabilidades entre las distintas capas del sistema. Los controladores administran la comunicación con el cliente, los servicios implementan la lógica de negocio y los repositorios gestionan el acceso a la base de datos. Esta organización permitió mantener un código ordenado y facilitó el desarrollo de cada funcionalidad de manera independiente.

7. Modelo de datos

El modelo de datos está compuesto por cuatro entidades principales. Producto representa cada artículo del catálogo, con sus atributos nombre, descripción, precio, stock y categoría.

Carrito agrupa los productos seleccionados por el usuario a través de ItemCarrito, entidad que asocia un producto con una cantidad determinada. Finalmente, Orden representa una compra confirmada y guarda una copia de los ítems junto con la fecha y hora, el total y un mensaje opcional del cliente.

De esta manera, el carrito se utiliza de forma temporal mientras el usuario prepara la compra, mientras que la orden conserva la información de forma permanente, independiente de cambios posteriores en el carrito. Las relaciones entre las entidades se establecen mediante anotaciones de JPA (@OneToMany y @ManyToOne), y el total se calcula como la suma del precio por la cantidad de cada ítem.

8. Diseño de la API REST

La API expone tres recursos principales: productos, carritos y órdenes. Todos los endpoints devuelven la información en formato JSON y utilizan los códigos de estado HTTP correspondientes a cada operación (por ejemplo, 201 al crear un recurso, 204 al eliminarlo, 400 ante datos inválidos y 404 cuando el recurso no existe). Las entradas se validan y los errores se devuelven de forma uniforme mediante el manejador global de excepciones. La siguiente tabla resume los endpoints implementados:

Método	Endpoint	Descripción	Códigos HTTP
GET	/api/productos	Listar el catálogo de productos	200
GET	/api/productos/{id}	Ver el detalle de un producto	200 / 404
POST	/api/productos	Crear un producto	201 / 400
PUT	/api/productos/{id}	Modificar un producto	200 / 400 / 404
DELETE	/api/productos/{id}	Eliminar un producto	204 / 404
POST	/api/carritos	Crear un carrito vacío	201
GET	/api/carritos/{id}	Ver el carrito con sus ítems	200 / 404
POST	/api/carritos/{id}/items	Agregar un ítem al carrito	200 / 400 / 404
PUT	/api/carritos/{id}/items/{itemId}	Modificar la cantidad de un ítem	200 / 400 / 404
DELETE	/api/carritos/{id}/items/{itemId}	Eliminar un ítem del carrito	200 / 404
POST	/api/ordenes/confirmar/{carritoid}	Confirmar el carrito como orden	201 / 400 / 404
GET	/api/ordenes	Listar el historial de órdenes	200
GET	/api/ordenes/{id}	Ver una orden	200 / 404

Antes de integrar el frontend, todos los endpoints fueron probados con Postman. La colección utilizada se incluye en el repositorio del proyecto.

9. Desarrollo del frontend

El frontend fue desarrollado con React, utilizando Vite como herramienta de desarrollo. Se construyó exclusivamente con componentes funcionales y se emplearon los hooks `useState` y `useEffect`: `useEffect` se utiliza para cargar la información inicial desde la API (productos, órdenes y carrito), mientras que `useState` administra el estado de la aplicación.

El flujo de datos sigue el patrón unidireccional de React. El estado principal, que incluye los productos, el carrito, las órdenes y la vista actual, se mantiene en el componente `App`, que lo transmite a los componentes hijos mediante props. Las acciones realizadas por el usuario se comunican nuevamente hacia el componente `App` a través de callbacks. Por ejemplo, la tarjeta de producto recibe la información del producto por props y, al presionar el botón de agregar al carrito, invoca una función provista por `App`.

La aplicación se organiza en componentes como `App` (estado general), `Header` (navegación), `Catalogo` y `ProductoCard` (listado y tarjetas de producto), `ProductoDetalle` (detalle de un producto), `ProductoForm` (alta y edición de productos), `Carrito` (ítems, cantidades y total) y `Pedidos` (historial de órdenes). Además, el identificador del carrito se guarda en el almacenamiento local del navegador para mantenerlo entre recargas, y todas las llamadas a la API se centralizan en un único módulo.

10. Integración y resolución de CORS

Problema: el frontend se ejecuta en `http://localhost:5173` (servidor de Vite) y el backend en `http://localhost:8080`. Al tratarse de orígenes distintos, el navegador aplica la política de mismo origen (Same-Origin) y bloquea las solicitudes del frontend hacia la API, impidiendo que la aplicación obtenga los datos.

Solución: para resolverlo se habilitó CORS en el backend mediante la anotación `@CrossOrigin` en los controladores de la API. De este modo, las respuestas incluyen las cabeceras necesarias (`Access-Control-Allow-Origin`) y el navegador permite la comunicación entre ambas partes. Con esta configuración, el frontend y el backend funcionan de manera integrada.

11. Declaración de uso de inteligencia artificial

Conforme a lo solicitado en la consigna, se declara el uso de herramientas de inteligencia artificial generativa como apoyo durante el desarrollo del trabajo. Estas herramientas se utilizaron en las siguientes tareas: la incorporación de validaciones, códigos de estado HTTP y el manejador global de excepciones en el backend; la generación de la colección de pruebas de Postman; la asistencia en la estructura de componentes y la integración del frontend; y la organización de algunas secciones de este informe. En todos los casos, el código y la documentación fueron revisados y comprendidos por el grupo, y cada integrante puede explicar las partes en las que trabajó.

Nota: ajustar esta declaración según las herramientas efectivamente utilizadas por el grupo (por ejemplo, ChatGPT, GitHub Copilot o Claude).

12. Conclusiones

El desarrollo de EcoMarket permitió construir una aplicación web funcional que integra un backend desarrollado con Spring Boot y un frontend implementado en React. A lo largo del proyecto se logró desarrollar un sistema capaz de gestionar un catálogo de productos, administrar un carrito de compras y registrar órdenes, manteniendo una estructura organizada y una correcta comunicación entre los distintos componentes de la aplicación.

Uno de los aspectos más importantes fue mantener una arquitectura por capas, separando la lógica de presentación, la lógica de negocio y el acceso a los datos. Esta organización permitió obtener un código más claro, reducir el acoplamiento entre las distintas partes del sistema y facilitar futuras modificaciones o ampliaciones del proyecto.

Durante el desarrollo también se implementaron buenas prácticas como el uso de DTO para el intercambio de información entre el frontend y el backend, la centralización del manejo de excepciones y la utilización de repositorios para el acceso a la base de datos. Estas decisiones contribuyeron a mantener una aplicación más ordenada y sencilla de mantener.

Como resultado se obtuvo una aplicación estable que cumple con los objetivos planteados inicialmente y permite realizar el flujo completo de una compra, desde la consulta de productos hasta la generación de una orden. Si bien el proyecto puede ampliarse incorporando nuevas funcionalidades, la base desarrollada proporciona una estructura sólida sobre la cual continuar trabajando.

En términos generales, EcoMarket representó una oportunidad para desarrollar una aplicación completa aplicando tecnologías ampliamente utilizadas en el desarrollo web

actual y poniendo en práctica criterios de organización, mantenimiento y diseño de software que resultan fundamentales en proyectos de mayor escala.

13. Reflexión grupal

El desarrollo de EcoMarket nos permitió integrar en un proyecto real los conceptos vistos en la materia, especialmente la arquitectura por capas y la comunicación entre un frontend y un backend mediante una API REST. Aprendimos a dividir responsabilidades, a probar los endpoints con Postman antes de integrarlos y a resolver problemas concretos como la configuración de CORS.

Si volviéramos a encarar el proyecto, planificaríamos con mayor anticipación el modelo de datos y la división de tareas, de modo de poder trabajar más en paralelo desde el inicio. El principal valor del trabajo fue la coordinación del equipo: nos repartimos el backend, el frontend y la documentación, y fuimos integrando cada parte de manera incremental hasta lograr una aplicación funcional y completa.